

# DW\_asymfifoctl\_s1\_sf

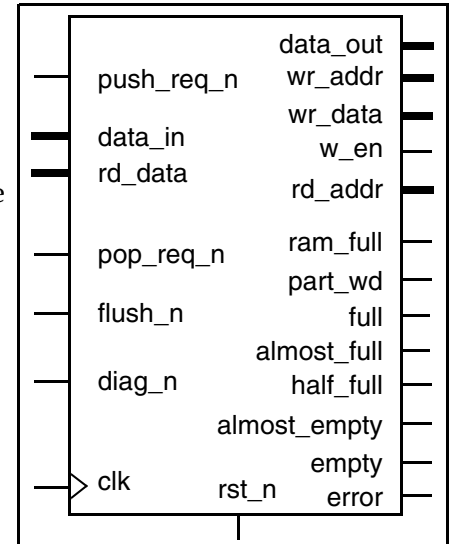
Asym. I/O Synch. (Single Clock) FIFO Controller - Static Flags

Version, STAR, and myDesignWare Subscriptions: [IP Directory](#)

## Features and Benefits

- Fully registered synchronous address and flag output ports
- All operations execute in a single clock cycle
- FIFO empty, half full, and full flags
- Asymmetric input and output bit widths (must be integer-multiple relationship)
- Word integrity flag for  $data\_in\_width < data\_out\_width$
- Flushing out partial word for  $data\_in\_width < data\_out\_width$
- Parameterized byte order within a word
- FIFO error flag indicating underflow, overflow, and pointer corruption
- Parameterized word depth
- Parameterized almost full and almost empty flags
- Parameterized reset mode (synchronous or asynchronous)
- Interfaces to common hard macro or compiled ASIC dual-port synchronous RAMs
- Includes a low-power implementation (at a sub-level) that has power benefits from minPower optimization (for details, see [Table 1-3](#) on page 3)

## Revision History



## Description

DW\_asymfifoctl\_s1\_sf is a FIFO RAM controller designed to interface with a dual-port synchronous RAM.

**Table 1-1 Pin Description**

| Pin Name   | Width | Direction | Function                                                                                                                                                                 |
|------------|-------|-----------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| clk        | 1 bit | Input     | Input clock                                                                                                                                                              |
| rst_n      | 1 bit | Input     | Reset input, active low <ul style="list-style-type: none"><li>■ Asynchronous if <math>rst\_mode = 0</math></li><li>■ Synchronous if <math>rst\_mode = 1</math></li></ul> |
| push_req_n | 1 bit | Input     | FIFO push request, active low                                                                                                                                            |
| flush_n    | 1 bit | Input     | Flushes the partial word into memory (fills in 0's) (for $data\_in\_width < data\_out\_width$ only)                                                                      |

**Table 1-1 Pin Description (Continued)**

| Pin Name     | Width                                                        | Direction | Function                                                                                               |
|--------------|--------------------------------------------------------------|-----------|--------------------------------------------------------------------------------------------------------|
| pop_req_n    | 1 bit                                                        | Input     | FIFO pop request, active low                                                                           |
| diag_n       | 1 bit                                                        | Input     | Diagnostic control, active low (for <i>err_mode</i> = 0, NC for other <i>err_mode</i> values)          |
| data_in      | <i>data_in_width</i> bits                                    | Input     | FIFO data to push                                                                                      |
| rd_data      | $\max(\text{data\_in\_width}, \text{data\_out\_width})$ bits | Input     | RAM data input to FIFO controller                                                                      |
| w_en         | 1 bit                                                        | Output    | Write enable output for write port of RAM, active low                                                  |
| empty        | 1 bit                                                        | Output    | FIFO empty output, active high                                                                         |
| almost_empty | 1 bit                                                        | Output    | FIFO almost empty output, active high, asserted when FIFO level $\leq ae\_level$                       |
| half_full    | 1 bit                                                        | Output    | FIFO half full output, active high                                                                     |
| almost_full  | 1 bit                                                        | Output    | FIFO almost full output, active high, asserted when FIFO level $\geq (\text{depth} - af\_level)$       |
| full         | 1 bit                                                        | Output    | FIFO full output, active high                                                                          |
| ram_full     | 1 bit                                                        | Output    | RAM full output, active high                                                                           |
| error        | 1 bit                                                        | Output    | FIFO error output, active high                                                                         |
| part_wd      | 1 bit                                                        | Output    | Partial word, active high (for <i>data_in_width</i> < <i>data_out_width</i> only; otherwise, tied low) |
| wr_data      | $\max(\text{data\_in\_width}, \text{data\_out\_width})$ bits | Output    | FIFO controller output data to RAM                                                                     |
| wr_addr      | $\text{ceil}(\log_2[\text{depth}])$ bits                     | Output    | Address output to write port of RAM                                                                    |
| rd_addr      | $\text{ceil}(\log_2[\text{depth}])$ bits                     | Output    | Address output to read port of RAM                                                                     |
| data_out     | <i>data_out_width</i> bits                                   | Output    | FIFO data to pop                                                                                       |

Table 1-2 Parameter Description

| Parameter      | Values               | Description                                                                                                                                                                                                                                                                                                                                                                              |
|----------------|----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| data_in_width  | 1 to 8192            | Width of the <code>data_in</code> bus. Values for <code>data_in_width</code> must be in an integer-multiple relationship with <code>data_out_width</code> . That is, either: <ul style="list-style-type: none"> <li>■ <math>data\_in\_width = K \times data\_out\_width</math>, or</li> <li>■ <math>data\_out\_width = K \times data\_in\_width</math></li> </ul> Where K is an integer. |
| data_out_width | 1 to 8192            | Width of the <code>data_out</code> bus. <code>data_out_width</code> must be in an integer-multiple relationship with <code>data_in_width</code> . That is, either: <ul style="list-style-type: none"> <li>■ <math>data\_in\_width = K \times data\_out\_width</math>, or</li> <li>■ <math>data\_out\_width = K \times data\_in\_width</math></li> </ul> Where K is an integer.           |
| depth          | 2 to 16777216        | Number of memory elements used in the FIFO                                                                                                                                                                                                                                                                                                                                               |
| ae_level       | 1 to $depth - 1$     | Almost empty level (the number of words in the FIFO at or below which the <code>almost_empty</code> flag is active)                                                                                                                                                                                                                                                                      |
| af_level       | 1 to $depth - 1$     | Almost full level (the number of empty memory locations in the FIFO at which the <code>almost_full</code> flag is active (see <a href="#">Figure 1-3</a> on page 16))                                                                                                                                                                                                                    |
| err_mode       | 0 to 2<br>Default: 1 | Error mode <ul style="list-style-type: none"> <li>■ 0: underflow/overflow with pointer latched checking</li> <li>■ 1: underflow/overflow latched checking</li> <li>■ 2: underflow/overflow unlatched checking</li> </ul>                                                                                                                                                                 |
| rst_mode       | 0 or 1<br>Default: 1 | Reset mode <ul style="list-style-type: none"> <li>■ 0: asynchronous reset</li> <li>■ 1: synchronous reset</li> </ul>                                                                                                                                                                                                                                                                     |
| byte_order     | 0 or 1<br>Default: 0 | Order of bytes or subword [ $subword < 8 \text{ bits} > subword$ ] within a word <ul style="list-style-type: none"> <li>■ 0: first byte is in most significant bits position</li> <li>■ 1: first byte is in the least significant bits position</li> </ul>                                                                                                                               |

Table 1-3 Synthesis Implementations

| Implementation Name | Function        | License Feature Required |
|---------------------|-----------------|--------------------------|
| str <sup>a</sup>    | Synthesis model | DesignWare <sup>b</sup>  |

- a. To achieve low-power benefits in sub-module implementations, you need to enable minPower; for details, see [“Enabling minPower”](#) on page 21.
- b. For releases prior to P-2019.03, the DesignWare-LP license feature is required to achieve low-power benefits.

**Table 1-4 Simulation Models**

| Model                                    | Function                             |
|------------------------------------------|--------------------------------------|
| DW03.DW_ASYMFIFOCTL_S1_SF_CFG_SIM        | Design unit name for VHDL simulation |
| dw/dw03/src/DW_asymfifoctl_s1_sf_sim.vhd | VHDL simulation model source code    |
| dw/sim_ver/DW_asymfifoctl_s1_sf.v        | Verilog simulation model source code |

**Table 1-5 Error Modes**

| err_mode | Error Types Detected                      | Error Output | diag_n    |
|----------|-------------------------------------------|--------------|-----------|
| 0        | Underflow/Overflow and Pointer Corruption | Latched      | Connected |
| 1        | Underflow/Overflow                        | Latched      | N/C       |
| 2        | Underflow/Overflow                        | Not Latched  | N/C       |

The input data bit width of DW\_asymfifoctl\_s1\_sf can be different than its output data bit width, but must have an integer-multiple relationship (the input bit width being a multiple of the output bit width or vice versa). In other words, either of the following conditions must be true:

- The  $data\_in\_width = K \times data\_out\_width$ , or
- The  $data\_out\_width = K \times data\_in\_width$

where  $K$  is a positive integer.

The RAM must have:

- A synchronous write port,
- Either asynchronous or synchronous read port, and
- The bit width must be the maximum of  $data\_in\_width$  or  $data\_out\_width$ .

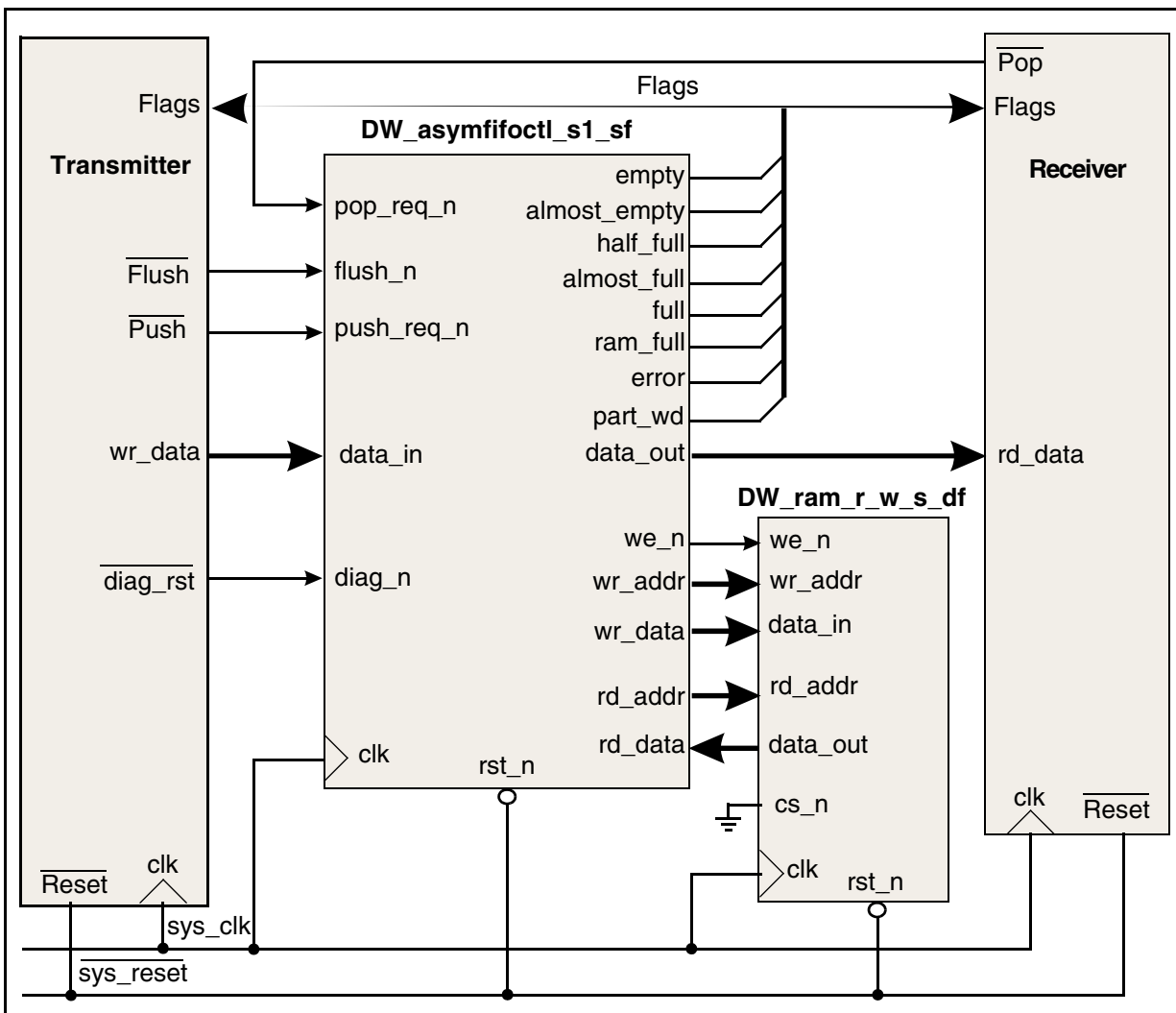
The asymmetric FIFO controller provides address generation, write-enable logic, flag logic, and operational error detection logic.

Parameterizable features include:

- FIFO depth (up to 24 address bits or 16,777,216 locations),
- Almost empty level,
- Almost full level,
- Level of error detection, and
- Type of reset (asynchronous or synchronous).

You specify these parameters when the controller is instantiated in the design.

Figure 1-1 on page 5 shows a typical application of the asymmetric FIFO controller.

**Figure 1-1 Example Usage of DW\_asymfifoctl\_s1\_sf**

## Writing to the FIFO (Push) When $\text{data\_in\_width} > \text{data\_out\_width}$

For cases where  $\text{data\_in\_width} > \text{data\_out\_width}$  (assuming that  $\text{data\_in\_width} = K \times \text{data\_out\_width}$ , where  $K$  is an integer larger than 1):

- The `flush_n` input pin is not used (at the system level, this pin should not be connected so that it is removed upon synthesis),
- The `part_wd` output pin is tied low, and
- The `data_in` bus is connected directly to the `wr_data` output bus.

For more information, see “Timing Waveforms” on page 18.

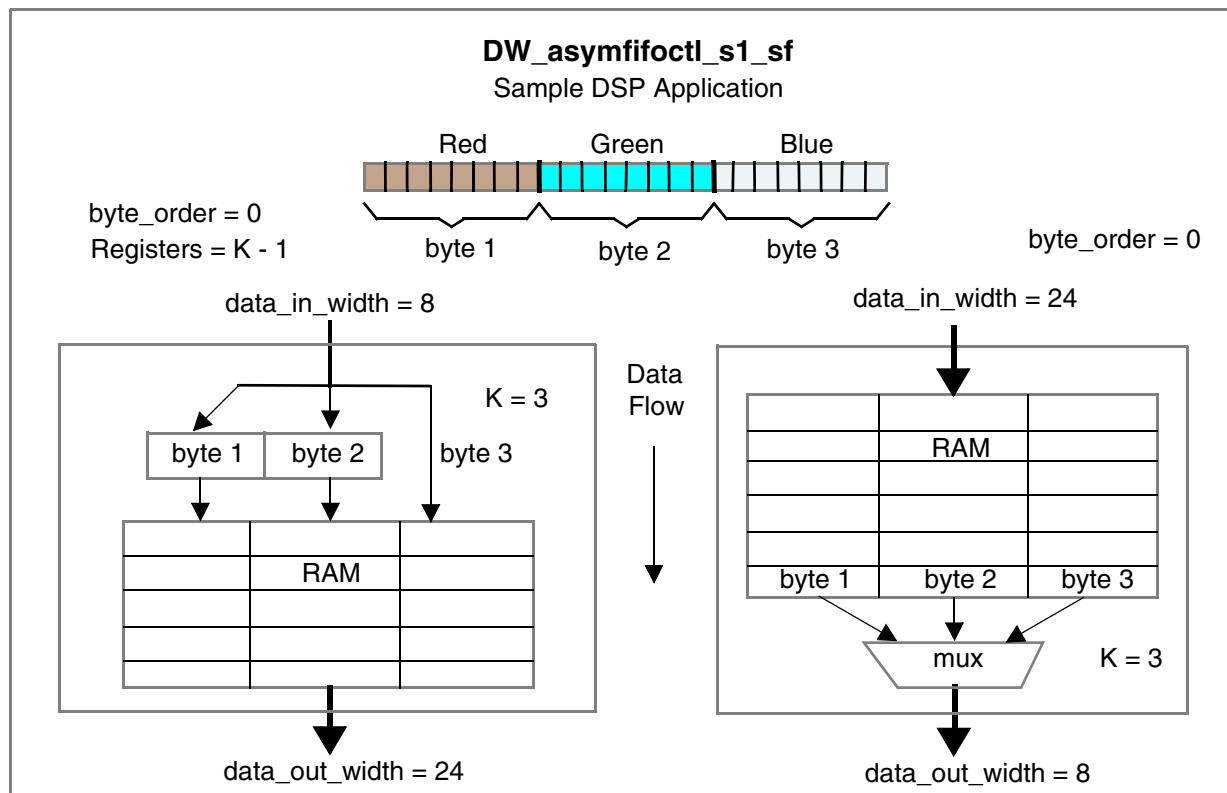
The `wr_addr` and `we_n` output ports of the FIFO controller provide the write address and synchronous write enable to the FIFO.

A push is executed when the `push_req_n` input is asserted (low) and the `full` flag is inactive (low) at the rising edge of `clk`. Asserting `push_req_n` when the `full` flag is inactive causes the following events to occur:

- The `we_n` is asserted immediately, preparing for a write to the RAM on the next clock, and
- On the next rising edge of `clk`, `wr_addr` is incremented.

Thus, the RAM is written and `wr_addr` (which always points to the address of the next word to be pushed) is incremented on the same rising edge of `clk`—the first clock after `push_req_n` is asserted. This means that `push_req_n` must be asserted early enough to propagate through the FIFO controller to the RAM before the next clock.

**Figure 1-2 Example of Asymmetric FIFO Controller Operation**



## Write Errors

An error occurs if a push is attempted while the FIFO is full. That is, if:

- The `push_req_n` input is asserted (low),
- The `full` flag is active (high), and
- The `pop_req_n` input is inactive (high), or there is more than one byte (or subword) left in the output buffer.

You should not use the DW\_asymfifoctrl\_s1\_sf to perform a simultaneous push and pop when the RAM is full. For details, see [“Simultaneous Push and Pop When data\\_in\\_width > data\\_out\\_width”](#) on page 12.

## Writing to the FIFO (Push) When data\_in\_width = data\_out\_width

In this case, the FIFO controller is a symmetric I/O FIFO controller. Its function is the same as DW\_fifoctrl\_s1\_sf, except for the `part_wd`, `flush`, and `ram_full` pins, which are unused.

The `wr_addr` and `we_n` output ports of the FIFO controller provide the write address and synchronous write enable to the FIFO.

The `data_in` bus is connected directly to the `wr_data` bus, and the `data_out` bus is connected directly to the `rd_data` bus.

A push is executed when the `push_req_n` input is asserted (low), and either:

- The `full` flag is inactive (low),

or,

- The `full` flag is active (high), and
- The `pop_req_n` input is asserted (low).

Thus, a push can occur even if the FIFO is full, as long as a pop is executed in the same cycle.

Asserting `push_req_n` in either of the above cases causes the data at the `data_in` port to be written to the next available location in the FIFO. This write occurs on the `clk` following the assertion of `push_req_n`. The data at the `data_in` port must be stable for a setup time before the rising edge of `clk`.

## Write Errors

An error occurs if a push is attempted while the FIFO is full. That is, if:

- The `push_req_n` input is asserted (low),
- The `full` flag is active (high), and
- The `pop_req_n` input is inactive (high).

## Writing to the FIFO (Push) When $\text{data\_in\_width} < \text{data\_out\_width}$

For cases where  $\text{data\_in\_width} < \text{data\_out\_width}$  (assuming that  $\text{data\_out\_width} = K \times \text{data\_in\_width}$ , where  $K$  is an integer larger than 1), every byte (or subword) written to the FIFO is first assembled into a full word with  $\text{data\_out\_width}$  bits. For more information, see “Timing Waveforms” on page 18.

The `wr_addr` and `we_n` output ports of the FIFO controller provide the write address and synchronous write enable to the FIFO.

A push of the partial word is executed when the `push_req_n` input is asserted (low), and either:

- The `full` flag is inactive (low),

or,

- The `full` flag is active (high), and
- The `pop_req_n` input is asserted (low)

at the rising edge of `clk`. Thus, a push can occur even if the FIFO is full, as long as a pop is executed in the same cycle.

For every byte (or subword) to be written, `push_req_n` toggles. Asserting `push_req_n`  $K$  times in either of the cases that enables a push causes the word accumulated in the input buffer (the first  $K - 1$  bytes are registered, the last byte is not; see Figure 1-3 on page 16.) to be written to the next available location in the FIFO memory. This write occurs on the `clk` following the assertion of `push_req_n`.

The order of bytes within a word is determined by the `byte_order` parameter.

The data at the `data_in` port must be stable for a setup time before the rising edge of `clk`, and `push_req_n` must be asserted early enough to propagate through the FIFO controller to the RAM before the next clock.

In this way, the RAM is written, and `wr_addr` (which always points to the address of the next word to be pushed) is incremented on the same rising edge of `clk`—the first clock after `push_req_n` is asserted  $K$  times.

## Partial Words

When a partial word is in the input buffer register, output flag `part_wd` is active (high). After  $K$  times pushing,  $K$  bytes (or subwords) are assembled into a full word ( $K - 1$  bytes in the input buffer register and the last byte on the `data_in` bus) by a combinational circuit. This achieves single clock cycle operation for the asymmetric FIFO controller. This full word is then written into memory. When a full word is sent from the input buffer into memory, `part_wd` goes inactive (low).

The order of bytes within a word is parameterized by parameter `byte_order`.

## Flushing the RAM

A flush feature is provided for the  $data\_in\_width < data\_out\_width$  case. This feature pushes a partial word into memory when there are less than  $K$  bytes accumulated in the input buffer. The input buffer is cleared after a flush.

A flush is allowed:

- When  $N$  bytes have been read since the last complete word (where  $0 < N < K$ ), and
- The sender device has no byte (or subword) to send at this moment,

while

- The higher level system requires that the receiver device be able to read these  $N$  bytes of data (from memory) without waiting,

or,

- For data byte word alignment.

The sender device activates `flush_n` so that the  $N$  bytes data is pushed into memory without waiting for a complete word to be assembled.

When the receiver reads the partial word from memory, the “leftover” bytes of the partial word ( $K - N$ ) are filled with 0s.

A flush is executed when the `flush_n` input is asserted (low), and either:

- The `ram_full` flag is inactive (low),

or,

- The `ram_full` flag is active (high), and
- The `pop_req_n` input is asserted (low)

at the rising edge of `clk`.

Asserting `flush_n` in either of the above cases causes the partial word accumulated in the input buffer to be written to the next available location in the FIFO memory. This write occurs on the `clk` following the assertion of `flush_n`.

Flushing the FIFO when the input buffer is empty (when the `part_wd` flag is inactive) is a “null” operation and does not cause an error.

## Simultaneous Flush and Push, and Flush and Pop

Flush can occur at the same time as a push. When `flush_n` and `push_req_n` are active at the same time, the FIFO:

- Flushes the partial word in the input buffer, if any, into the memory, and
- Pushes the byte in the `data_in` bus into the input buffer

in the same clock cycle.

A flush can occur at the same time as a pop when the FIFO is not empty, even when the FIFO is full. For a detailed description, refer to “[Reading from the FIFO \(Pop\) When  \$data\\_in\\_width < data\\_out\\_width\$](#) ” on page 11.

## Write Errors

An error occurs if a push is attempted while the FIFO is full. That is, if:

- The `push_req_n` input is active (low),
- The `empty` flag is active (high), and
- The `pop_req_n` input is inactive (high).

## Reading from the FIFO (Pop) When `data_in_width > data_out_width`

For cases where `data_in_width > data_out_width` (assuming that `data_in_width = K × data_out_width`, where  $K$  is an integer larger than 1), the number of bits in a word stored in memory is `data_in_width`. The bit width for each out-going byte (or subword) is `data_out_width`.

For every byte (or subword) to be read, `pop_req_n` toggles. Each pop causes one byte (or subword) to be read. Toggling `pop_req_n`  $K$  times results in one full word (`data_in_width` bits) being read. The order of the output bytes within a word is determined by the `byte_order` parameter.

The read port of the memory can be either synchronous or asynchronous. In either case, the `data_out` output port of the DW\_asymfifoc1\_s1\_sf provides prefetchable data (the next byte of memory read data to be read) to the output logic.

For RAMs with a synchronous read port, the output data is captured in the output stage of the memory. For RAMs with an asynchronous read port, the output data is captured by the next stage of logic after the FIFO.

A pop operation occurs when `pop_req_n` is asserted (low) when the FIFO is not empty. Asserting `pop_req_n` when the output buffer is not empty causes the `data_out` output port to be switched to the next byte (or subword) on the next rising edge of `clk`. Thus, memory read data must be captured on the `clk` following the assertion of `pop_req_n`.

For more information, see [“Timing Waveforms”](#) on page 18.

## Read Errors

An error occurs if:

- The `pop_req_n` input is active (low), and
- The `empty` flag is active (high).

## Reading from the FIFO (Pop) When $\text{data\_in\_width} = \text{data\_out\_width}$

In this case, the FIFO controller is a symmetric FIFO controller. Its function is the same as the DW\_fifoctrl\_s1\_sf, except for the `part_wd`, `flush`, and `ram_full` pins, which are unused.

The `data_in` bus is connected directly to the `wr_data` bus, and the `data_out` bus is connected directly to the `rd_data` bus.

The read port of the RAM can be either synchronous or asynchronous. In either case, the `rd_addr` output port of the DW\_asymfifoctrl\_s1\_sf provides the read address to the RAM. The `rd_addr` output bus always points to, thus prefetches, the next word of RAM read data to be popped.

For RAMs with a synchronous read port, the output data is captured in the output stage of the RAM. For RAMs with an asynchronous read port, the output data is captured by the next stage of logic after the FIFO.

A pop operation occurs when `pop_req_n` is asserted (low), as long as the FIFO is not empty (empty output low). Asserting `pop_req_n` causes the `rd_addr` pointer to be incremented on the next rising edge of `clk`. Thus, the RAM read data must be captured on the `clk` following the assertion of `pop_req_n`.

For more about the pop operation for RAMs, see [“Timing Waveforms”](#) on page 18.

### Read Errors

An error occurs if:

- The `pop_req_n` input is active (low), and
- The `empty` flag is active (high).

## Reading from the FIFO (Pop) When $\text{data\_in\_width} < \text{data\_out\_width}$

For cases where  $\text{data\_in\_width} < \text{data\_out\_width}$  (assuming that  $\text{data\_out\_width} = K \times \text{data\_in\_width}$ , where  $K$  is an integer larger than 1), the number of bits in a word stored in memory is `data_out_width`. The `rd_data` bus is connected directly to the `data_out` bus.

The read port of the RAM can be either synchronous or asynchronous. In either case, the byte (or subword) to be read is available for prefetching at the FIFO `data_out` output port.

For RAMs with a synchronous read port, output data is captured in the output stage of the RAM. For RAMs with an asynchronous read port, output data is captured by the next stage of logic after the FIFO.

A pop operation occurs when `pop_req_n` is asserted (low), as long as the FIFO is not empty. The operation occurs on the next rising edge of `clk`. Thus, the RAM read data must be captured on the `clk` following the assertion of `pop_req_n`.

For more about the pop operation for RAMs, see [“Timing Waveforms”](#) on page 18.

### Read Errors

An error occurs if:

- The `pop_req_n` input is active (low), and
- The `empty` flag is active (high).

## Simultaneous Push and Pop When $\text{data\_in\_width} > \text{data\_out\_width}$

You should not use the DW\_asymfifoctrl\_s1\_sf to perform a simultaneous push and pop when the RAM is full.

For  $\text{data\_in\_width} > \text{data\_out\_width}$  ( $\text{data\_in\_width} = K \times \text{data\_out\_width}$ ) cases, push and pop can occur at the same time if:

- The FIFO is neither full nor empty, or
- The FIFO is full but there is only one byte (or subword) in the output buffer.

With the FIFO neither full nor empty (both `full` and `empty` signals inactive), the byte to be read is available for prefetching at the FIFO `data_out` output port.

When `pop_req_n` and `push_req_n` are both asserted, the following events occur on the next rising edge of `clk`:

- Pop data is captured by the next stage of logic after the FIFO, and
- Write data is pushed into the location pointed to by `wr_addr`.

When the FIFO is full, a simultaneous push and pop can occur only if  $K - 1$  bytes of the word in the output buffer have been already read, and there is only one byte (or subword) left to be read in the output buffer; otherwise, simultaneous push and pop causes an overflow error; (see [Figure 1-3](#) on page 16).

There are no flags that indicate a valid or invalid condition for a simultaneous push and pop when the FIFO is full. Designers who want an indication of this condition should create the necessary logic external to the FIFO controller.

When the FIFO is empty, simultaneous push and pop causes an error, since there is no pop data to prefetch.

## Simultaneous Push and Pop When $\text{data\_in\_width} = \text{data\_out\_width}$

In this case, the FIFO controller is a symmetric FIFO controller. Its function is the same as DW\_fifoctrl\_s1\_sf, except for the `part_wd`, `flush`, and `ram_full` pins, which are unused. The `data_in` bus is connected directly to `wr_data`, and `rd_data` is connected directly to the `data_out` bus.

Push and pop can occur at the same time if there is data in the FIFO, even when the FIFO is full. With the FIFO not empty, `rd_addr` is pointing to the next address to be popped, and the pop data is available to be prefetched at the RAM output. When `pop_req_n` and `push_req_n` are both asserted, the following events occur on the next rising edge of `clk`:

- Pop data is captured by the next stage of logic after the FIFO, and
- The new data is pushed into the same location from which the data was popped.

Thus, there is no conflict in a simultaneous push and pop when the FIFO is full. A simultaneous push and pop cannot occur when the FIFO is empty, since there is no pop data to prefetch.

## Simultaneous Push and Pop When $\text{data\_in\_width} < \text{data\_out\_width}$

For  $\text{data\_in\_width} < \text{data\_out\_width}$  ( $\text{data\_out\_width} = K \times \text{data\_in\_width}$ ) cases, a push (or flush) and pop can occur at the same time if the FIFO is not empty. With the FIFO not empty (`empty` active), pop data is available to be prefetched at the FIFO (and the RAM) output.

When `pop_req_n` and `push_req_n` are both asserted, the following events occur on the next rising edge of `clk`:

- Pop data is captured by the next stage of logic after the FIFO,
- Write data is pushed into the input buffer, which may in turn be pushed into the next available memory location after  $K$  pushes, and
- For a flush, the partial word in the input buffer is pushed into the next available memory location. The input buffer is cleared after the flush.

For  $\text{data\_in\_width} < \text{data\_out\_width}$  cases, there is no conflict in a simultaneous push and pop when the FIFO is full, because the bit width of the outgoing word is larger than that of the incoming byte (or subword), and the incoming data speed is slower than the outgoing data speed.

When the FIFO is empty, a simultaneous push and pop causes an error, since there is no pop data to prefetch.

## Reset

### `rst_mode`

This parameter selects whether reset is:

- Asynchronous (`rst_mode = 0`), or
- Synchronous (`rst_mode = 1`).

If the asynchronous mode is selected, asserting `rst_n` (setting it low) immediately causes the:

- Internal address pointers to be set to 0,
- Input or output buffer to be reset, and
- Flags and error output to be initialized.

If the synchronous mode is selected, the internal address pointers, flags, and error outputs are initialized at the rising edge of `clk` after `rst_n` is asserted.

The error output and flags are initialized as follows:

- The `empty` and `almost_empty` are initialized to 1, and
- All other flags and the `error` output are initialized to 0.

## Errors

### **err\_mode**

The *err\_mode* parameter determines which possible fault conditions are detected, and whether the *error* output remains active until reset or only for the clock cycle in which the error was detected.

When the *err\_mode* parameter is set to 0 at design time, the *diag\_n* input provides an unconditional synchronous reset to the value of the *rd\_addr* output port. This can be used to intentionally cause the FIFO address pointers to become corrupted, forcing a pointer inconsistency-type error.

For normal operation when *err\_mode* = 0, *diag\_n* should be driven inactive (high). When the *err\_mode* parameter is set to 1 or 2, the *diag\_n* input is ignored (unconnected).

### **error**

The *error* output indicates a fault in the operation of the FIFO control logic. There are several possible causes for the *error* output to be activated:

1. Overflow (push with no pop while *full*; or, flush while *ram\_full* for *data\_in\_width* < *data\_out\_width* case; or, push when *full* is active and the output buffer has more than one byte for *data\_in\_width* > *data\_out\_width* case).
2. Underflow (pop while empty).
3. Empty pointer mismatch (*rd\_addr* ≠ *wr\_addr* when empty).
4. Full pointer mismatch (*rd\_addr* ≠ *wr\_addr* when full).
5. In between pointer mismatch (*rd\_addr* = *wr\_addr* when neither empty nor full).

When *err\_mode* = 0, all five causes are detected and the *error* output (once activated) remains active until reset. When *err\_mode* = 1, only causes 1 and 2 are detected, and the *error* output (once activated) remains active until reset. When *err\_mode* = 2, only causes 1 and 2 are detected, and the *error* output stays active only for the clock cycle in which the *error* is detected. For error mode descriptions, see [Table 1-5](#) on page 4. The *error* output is set low when *rst\_n* is applied.

## Controller Status Flag Outputs

The status flags are as follows. For operation of the status flags, see [Figure 1-3](#) on page 16.

### **empty**

The `empty` output indicates that there are no words or bytes in the FIFO available to be popped. The `empty` output is set high when `rst_n` is applied.

### **almost\_empty**

The `almost_empty` output is asserted when there are no more than `ae_level` words currently in the FIFO available to be popped. The `ae_level` parameter defines the almost empty threshold. The `almost_empty` output is useful for preventing the FIFO from underflowing. The `almost_empty` output is set high when `rst_n` is applied.

### **half\_full**

The `half_full` output is active (high) when at least half the FIFO memory locations are occupied. The `half_full` output is set low when `rst_n` is applied.

### **almost\_full**

The `almost_full` output is asserted when there are no more than `af_level` empty locations in the FIFO. The `af_level` parameter defines the almost full threshold, and is useful for preventing the FIFO from overflowing. The `almost_full` output is set low when `rst_n` is applied.

### **full**

The `full` output indicates that the FIFO is full, and there is no space available for push data. The `full` output is set low when `rst_n` is applied.

### **ram\_full**

The `ram_full` output is used for the `data_in_width < data_out_width` case. The `ram_full` output indicates that the RAM is full, and there is no space available for flushing a partial word into the RAM. The `ram_full` output is set low when `rst_n` is applied.

For `data_in_width < data_out_width`, `ram_full` is tied to the `full` output.

### **part\_wd**

This flag is only used for the `data_in_width < data_out_width` case. The `part_wd` output indicates that the FIFO has a partial word accumulated in the input buffer. The `part_wd` output is set low when `rst_n` is applied.

For `data_in_width ≥ data_out_width`, `part_wd` is tied low, since the input data is always a full word.

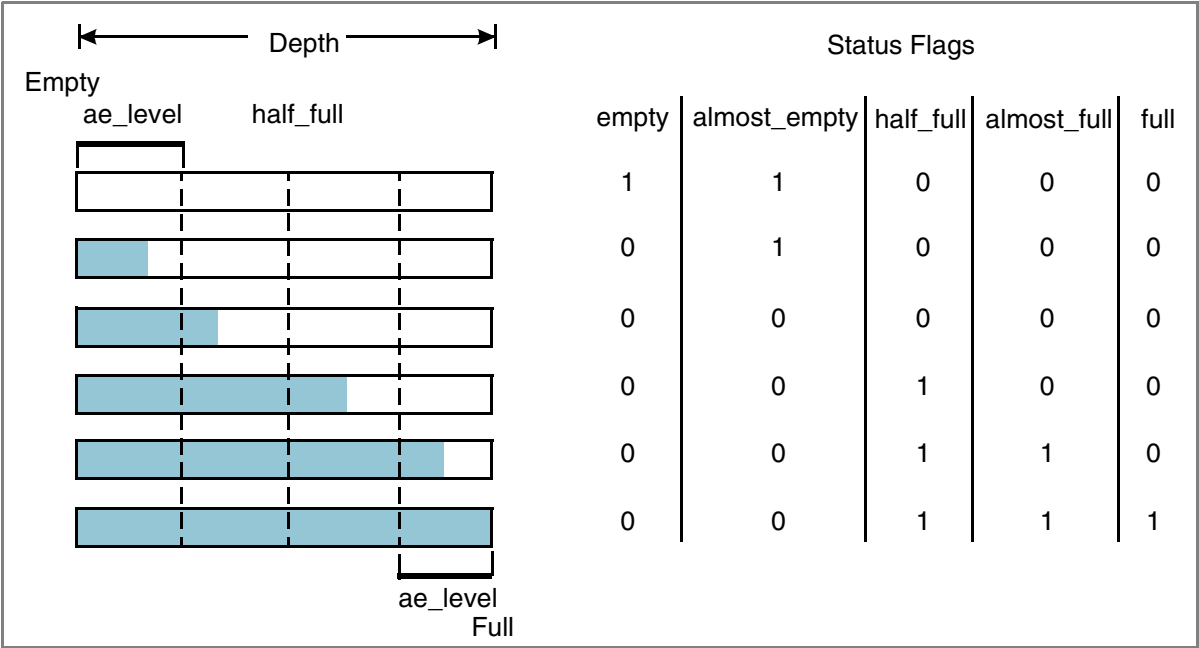
Application Notes

The ae\_level parameter value is chosen at design time to give the input flow control logic enough time to begin pushing data into the FIFO before the last word is popped by the output flow control logic.

The af\_level parameter value is chosen at design time to give the output flow control logic enough time to begin popping data out of the FIFO before the FIFO is full. In other situations, this time is needed to cause the input flow control logic to interrupt the pushing of data into the FIFO.

Figure 1-3 shows the status flags of the DW\_asymfifoctl\_s1\_sf FIFO at various FIFO storage levels.

Figure 1-3 DW\_asymfifoctl\_s1\_sf FIFO Status Flags



## Suppressing Warning Messages During Verilog Simulation

The Verilog simulation model includes macros that allow you to suppress warning messages during simulation.

To suppress all warning messages for all DWBB components, define the DW\_SUPPRESS\_WARN macro in either of the following ways:

- Specify the Verilog preprocessing macro in Verilog code:

```
`define DW_SUPPRESS_WARN
```

- Or, include a command line option to the simulator, such as:

```
+define+DW_SUPPRESS_WARN (which is used for the Synopsys VCS simulator)
```

The warning messages for this model include the following:

- If values other than 1 or 0 are present on a clock port, the following message is displayed:

```
WARNING: <instance_path>.<clock_name>_monitor:  
at time = <timestamp>, Detected unknown value, x, on <clock_name> input.
```

To suppress only this warning message for all DWBB components, use the following macro:

- Define the DW\_DISABLE\_CLK\_MONITOR macro. You can define this macro in the following ways:

- Specify the Verilog preprocessing macro in Verilog code:

```
`define DW_DISABLE_CLK_MONITOR
```

- Or, include a command line option to the simulator, such as:

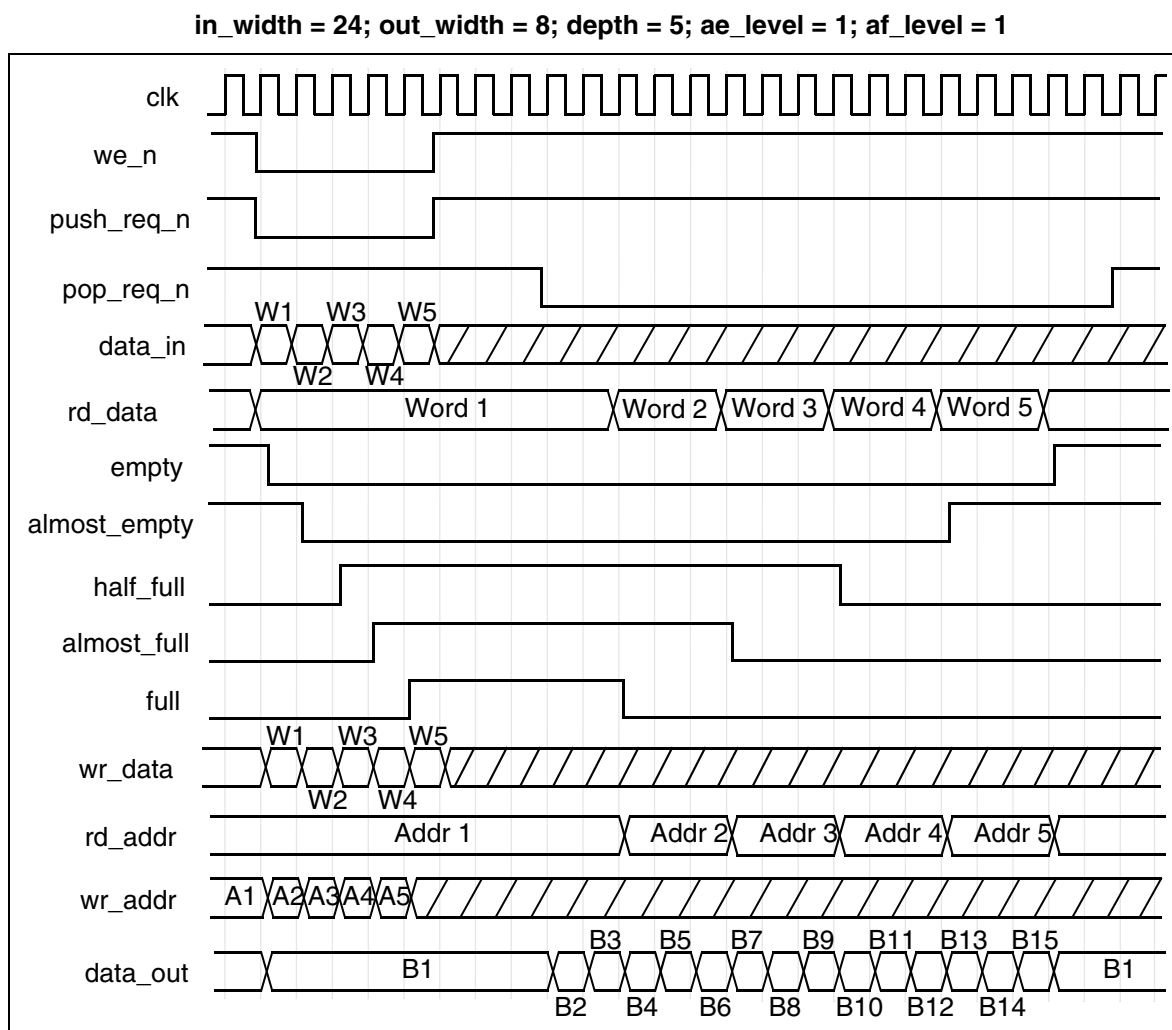
```
+define+DW_DISABLE_CLK_MONITOR (which is used for the Synopsys VCS simulator)
```

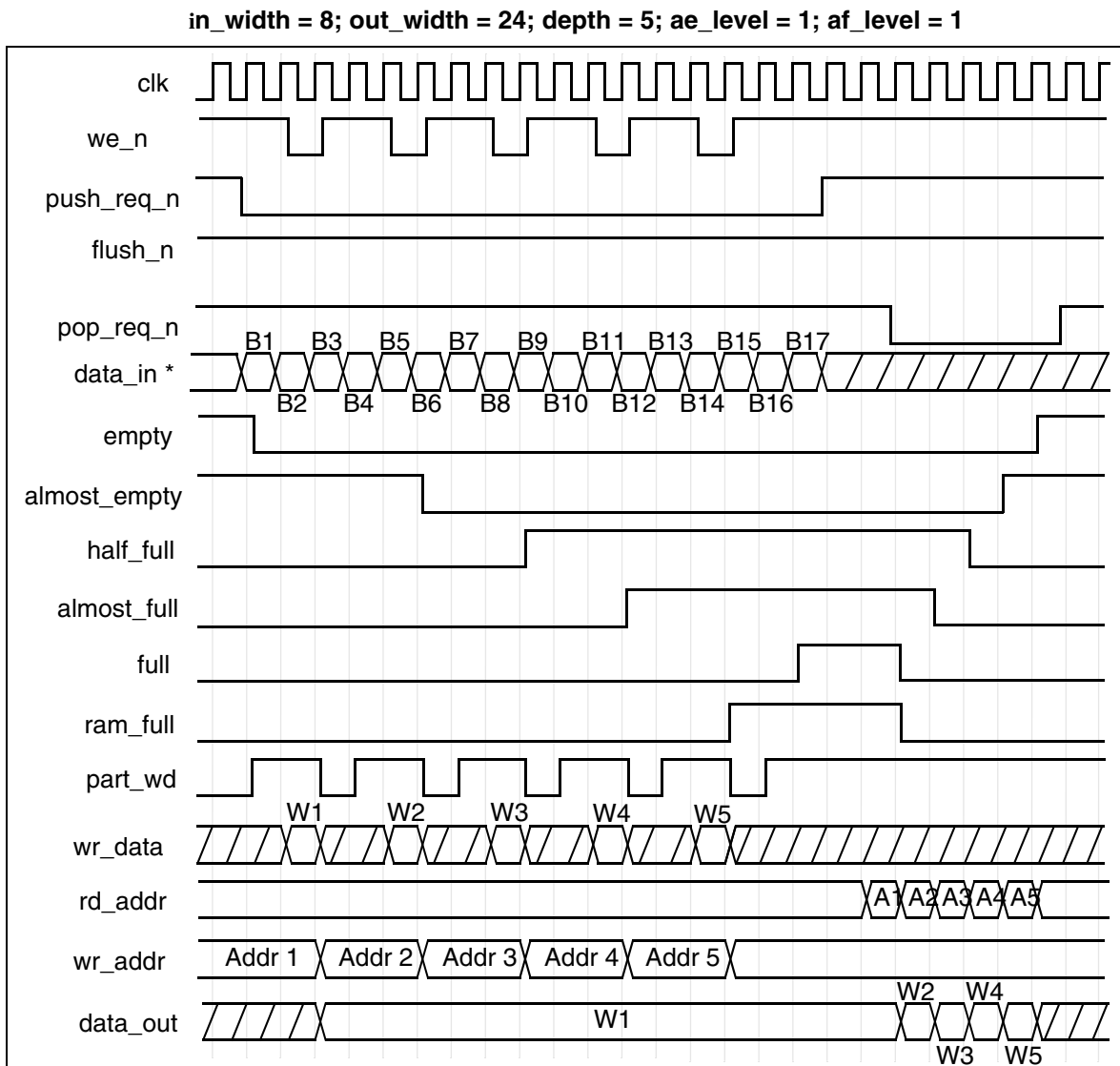
This message is also suppressed using the DW\_SUPPRESS\_WARN macro explained earlier.

## Timing Waveforms

The following figures show timing diagrams for various conditions of DW\_asymfifoctl\_s1\_sf.

**Figure 1-4 Status Flag Timing Waveforms for data\_in\_width > data\_out\_width**



**Figure 1-5 Status Flag Timing Waveforms for data\_in\_width < data\_out\_width**

\* Note: B16 and B17 are the first two slices of what would be W6 (not shown). B16 and B17 are waiting in a 2-stage assembly buffer in this case, as shown in [Figure 1-2](#) on page 6 with byte1 and byte2, respectively, for  $\text{data\_in\_width} = 8$ .

Figure 1-6 Status Flag Timing Waveforms for Flush Operation

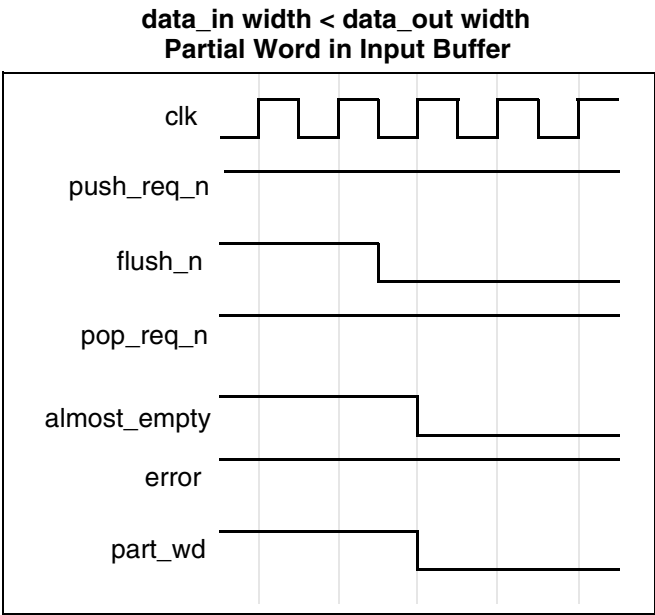
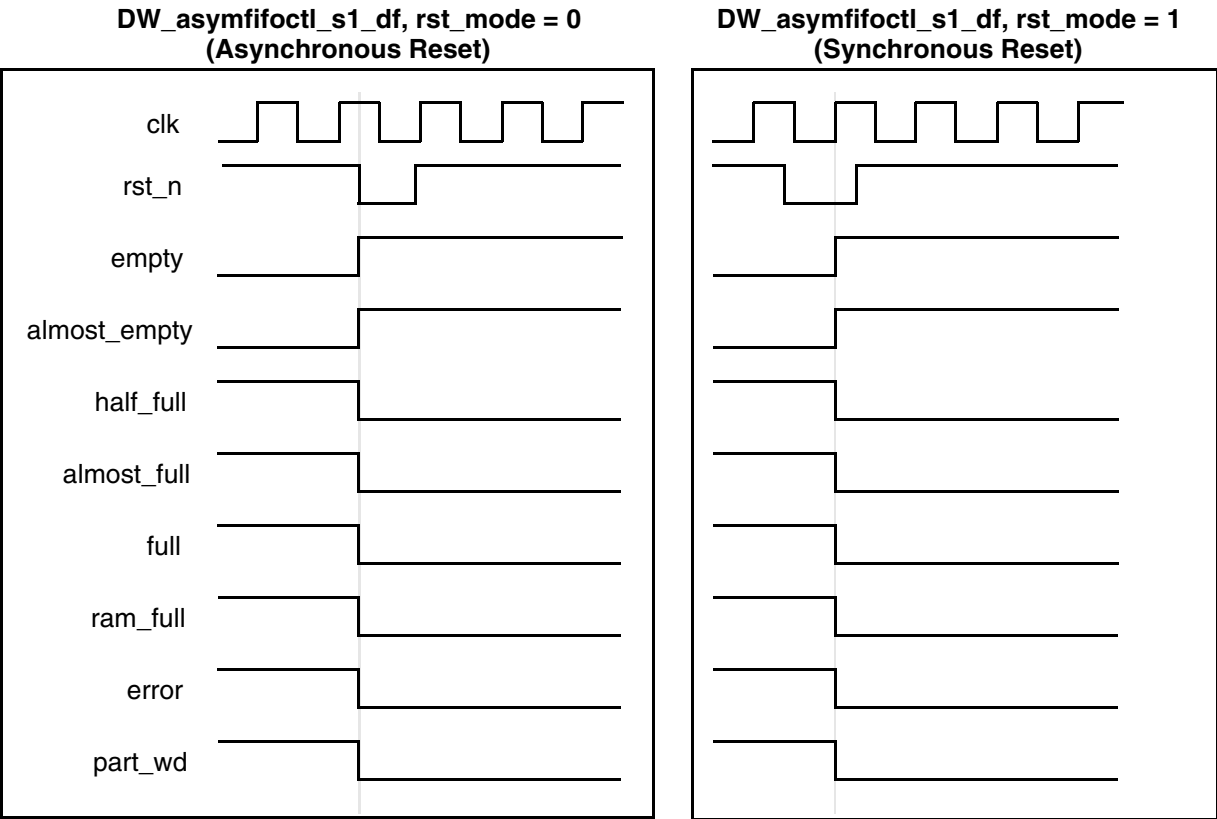


Figure 1-7 Reset Timing Waveforms



## Enabling minPower

You can instantiate this component without enabling minPower, but to achieve power savings from the low-power implementation (at a sub-level--see [Table 1-3](#) on page 3), you must enable minPower optimization, as follows:

- Design Compiler

- Version P-2019.03 and later:

- ```
set power_enable_minpower true
```

- Before version P-2019.03 (requires the DesignWare-LP license feature):

- ```
set synthetic_library {dw_foundation.sldb dw_minpower.sldb}  
set link_library {* $target_library $synthetic_library}
```

- Fusion Compiler

Optimization for minPower is enabled as part of the total\_power metric setting. To enable the total\_power metric, use the following:

```
set_qor_strategy -stage synthesis -metric total_power
```

## Related Topics

- [Memory – FIFO Overview](#)
- [DesignWare Building Block IP User Guide](#)

## HDL Usage Through Component Instantiation - VHDL

```
library IEEE,DWARE;
use IEEE.std_logic_1164.all;
use DWARE.DWpackages.all;
use DWARE.DW_foundation_comp.all;

entity DW_asymfifoc1_s1_sf_inst is
  generic (inst_data_in_width  : INTEGER := 8;
           inst_data_out_width : INTEGER := 16;
           inst_depth          : INTEGER := 8;
           inst_ae_level       : INTEGER := 4;
           inst_af_level       : INTEGER := 4;
           inst_err_mode        : INTEGER := 1;
           inst_rst_mode        : INTEGER := 1;
           inst_byte_order      : INTEGER := 0 );
  port (inst_clk      : in std_logic;
        inst_rst_n    : in std_logic;
        inst_push_req_n : in std_logic;
        inst_flush_n  : in std_logic;
        inst_pop_req_n : in std_logic;
        inst_diag_n   : in std_logic;
        inst_data_in  : in std_logic_vector(inst_data_in_width-1 downto 0);
        inst_rd_data  : in std_logic_vector(maximum(inst_data_in_width,
                                                    inst_data_out_width)-1 downto 0);

        we_n_inst      : out std_logic;
        empty_inst     : out std_logic;
        almost_empty_inst : out std_logic;
        half_full_inst : out std_logic;
        almost_full_inst : out std_logic;
        full_inst      : out std_logic;
        ram_full_inst  : out std_logic;
        error_inst     : out std_logic;
        part_wd_inst   : out std_logic;
        wr_data_inst   : out std_logic_vector(maximum(inst_data_in_width,
                                                    inst_data_out_width)-1 downto 0);
        wr_addr_inst  : out std_logic_vector(bit_width(inst_depth)-1 downto 0);
        rd_addr_inst  : out std_logic_vector(bit_width(inst_depth)-1 downto 0);
        data_out_inst : out std_logic_vector(inst_data_out_width-1 downto 0) );
end DW_asymfifoc1_s1_sf_inst;
```

architecture inst of DW\_asymfifoc1\_s1\_sf\_inst is  
begin

-- Instance of DW\_asymfifoc1\_s1\_sf

U1 : **DW\_asymfifoc1\_s1\_sf**

```
generic map (data_in_width => inst_data_in_width,
             data_out_width => inst_data_out_width, depth => inst_depth,
             ae_level => inst_ae_level,   af_level => inst_af_level,
             err_mode => inst_err_mode,   rst_mode => inst_rst_mode,
             byte_order => inst_byte_order )
```

```
port map (clk => inst_clk,   rst_n => inst_rst_n,
          push_req_n => inst_push_req_n,   flush_n => inst_flush_n,
          pop_req_n => inst_pop_req_n,   diag_n => inst_diag_n,
          data_in => inst_data_in,   rd_data => inst_rd_data,
          we_n => we_n_inst,   empty => empty_inst,
          almost_empty => almost_empty_inst,
          half_full => half_full_inst,   almost_full => almost_full_inst,
          full => full_inst,   ram_full => ram_full_inst,
          error => error_inst,   part_wd => part_wd_inst,
          wr_data => wr_data_inst,   wr_addr => wr_addr_inst,
          rd_addr => rd_addr_inst,   data_out => data_out_inst );
```

end inst;

-- pragma translate\_off

configuration DW\_asymfifoc1\_s1\_sf\_inst\_cfg\_inst  
of DW\_asymfifoc1\_s1\_sf\_inst is

for inst

end for; -- inst

end DW\_asymfifoc1\_s1\_sf\_inst\_cfg\_inst;

-- pragma translate\_on

## HDL Usage Through Component Instantiation - Verilog

```
module DW_asymfifoc1_s1_sf_inst(inst_clk, inst_rst_n, inst_push_req_n,  
    inst_flush_n, inst_pop_req_n, inst_diag_n, inst_data_in, inst_rd_data,  
    we_n_inst, empty_inst, almost_empty_inst, half_full_inst, almost_full_inst,  
    full_inst, ram_full_inst, error_inst, part_wd_inst, wr_data_inst,  
    wr_addr_inst, rd_addr_inst, data_out_inst );  
  
    parameter data_in_width = 8;  
    parameter data_out_width = 16;  
    parameter depth = 8;  
    parameter ae_level = 4;  
    parameter af_level = 4;  
    parameter err_mode = 1;  
    parameter rst_mode = 1;  
    parameter byte_order = 0;  
    `define bit_width_depth 3 // ceil(log2(depth))  
  
    input inst_clk;  
    input inst_rst_n;  
    input inst_push_req_n;  
    input inst_flush_n;  
    input inst_pop_req_n;  
    input inst_diag_n;  
    input [data_in_width-1 : 0] inst_data_in;  
    input [((data_in_width > data_out_width)?  
        data_in_width : data_out_width)-1 : 0] inst_rd_data;  
    output we_n_inst;  
    output empty_inst;  
    output almost_empty_inst;  
    output half_full_inst;  
    output almost_full_inst;  
    output full_inst;  
    output ram_full_inst;  
    output error_inst;  
    output part_wd_inst;  
    output [((data_in_width > data_out_width)?  
        data_in_width : data_out_width)-1 : 0] wr_data_inst;  
    output [`bit_width_depth-1 : 0] wr_addr_inst;  
    output [`bit_width_depth-1 : 0] rd_addr_inst;  
    output [data_out_width-1 : 0] data_out_inst;
```

```
// Instance of DW_asymfifoctl_s1_sf
DW_asymfifoctl_s1_sf #(data_in_width, data_out_width, depth, ae_level,
                        af_level, err_mode, rst_mode, byte_order)
U1 (.clk(inst_clk),    .rst_n(inst_rst_n),    .push_req_n(inst_push_req_n),
    .flush_n(inst_flush_n),    .pop_req_n(inst_pop_req_n),
    .diag_n(inst_diag_n),    .data_in(inst_data_in),
    .rd_data(inst_rd_data),    .we_n(we_n_inst),    .empty(empty_inst),
    .almost_empty(almost_empty_inst),    .half_full(half_full_inst),
    .almost_full(almost_full_inst),    .full(full_inst),
    .ram_full(ram_full_inst),    .error(error_inst),
    .part_wd(part_wd_inst),    .wr_data(wr_data_inst),
    .wr_addr(wr_addr_inst),    .rd_addr(rd_addr_inst),
    .data_out(data_out_inst) );
endmodule
```

## Revision History

For notes about this release, see the [DesignWare Building Block IP Release Notes](#).

For lists of both known and fixed issues for this component, refer to the [STAR report](#).

For a version of this datasheet that shows the most recent changes, click [here](#).

| Date           | Release       | Updates                                                                                                                                                                                                                                                        |
|----------------|---------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| September 2024 | DWBB_202409.0 | <ul style="list-style-type: none"> <li>■ Raised maximum values for <i>data_in_width</i> and <i>data_out_width</i> in <a href="#">Table 1-2</a> on page <a href="#">3</a></li> </ul>                                                                            |
| March 2024     | DWBB_202312.2 | <ul style="list-style-type: none"> <li>■ Restored correct maximum values for <i>data_in_width</i> and <i>data_out_width</i> in <a href="#">Table 1-2</a> on page <a href="#">3</a></li> </ul>                                                                  |
| December 2023  | DWBB_202312.0 | <ul style="list-style-type: none"> <li>■ Updated date and version; no technical changes</li> </ul>                                                                                                                                                             |
| July 2020      | DWBB_201912.5 | <ul style="list-style-type: none"> <li>■ Adjusted content and title of “<a href="#">Suppressing Warning Messages During Verilog Simulation</a>” on page <a href="#">17</a> and added the DW_SUPPRESS_WARN macro</li> </ul>                                     |
| October 2019   | DWBB_201903.5 | <ul style="list-style-type: none"> <li>■ Added the “Disabling Clock Monitor Messages” section</li> </ul>                                                                                                                                                       |
| March 2019     | DWBB_201903.0 | <ul style="list-style-type: none"> <li>■ Clarified license requirements in <a href="#">Table 1-3</a> on page <a href="#">3</a></li> <li>■ Added “<a href="#">Enabling minPower</a>” on page <a href="#">21</a></li> </ul>                                      |
| January 2019   | DWBB_201806.5 | <ul style="list-style-type: none"> <li>■ Updated example in “<a href="#">HDL Usage Through Component Instantiation - VHDL</a>” on page <a href="#">22</a></li> </ul>                                                                                           |
| October 2017   | DWBB_201709.1 | <ul style="list-style-type: none"> <li>■ Replaced the synthesis implementations in <a href="#">Table 1-3</a> on page <a href="#">3</a> with the str implementation</li> <li>■ Added this Revision History table and the document links on this page</li> </ul> |

## Copyright Notice and Proprietary Information

© 2024 Synopsys, Inc. All rights reserved. This Synopsys software and all associated documentation are proprietary to Synopsys, Inc. and may only be used pursuant to the terms and conditions of a written license agreement with Synopsys, Inc. All other use, reproduction, modification, or distribution of the Synopsys software or the associated documentation is strictly prohibited.

### Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

### Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

### Trademarks

Synopsys and certain Synopsys product names are trademarks of Synopsys, as set forth at <https://www.synopsys.com/company/legal/trademarks-brands.html>.

All other product or company names may be trademarks of their respective owners.

### Free and Open-Source Software Licensing Notices

If applicable, Free and Open-Source Software (FOSS) licensing notices are available in the product installation.

### Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

Synopsys, Inc.  
[www.synopsys.com](http://www.synopsys.com)

